Calendar Page — Information Document

Page Overview

Purpose

The Calendar page gives a time-ordered view of the app's execution activity. It plots every historical execution on the day it ran and overlays the next scheduled fire times of every active routine, so the user can see past literature-surveillance work and upcoming automated runs in a single monthly, weekly, or daily grid.

Primary User Flows

- Open the page and see the current month populated with past executions (Deep Dive, Deep Sweep, routine-driven runs) plus the upcoming fire times of each active routine.
- Switch between `Month`, `Week`, and `Day` views via FullCalendar's built-in header toolbar; use the toolbar's `prev`, `next`, and `today` buttons to navigate time.
- Narrow the displayed events with the `Type` filter (Deep Dive / Deep Sweep / Routine), the `Status` filter, and the `Routines` multi-select dropdown (toggle individual routines, or use `Select all` / `Select none`).
- Click any event to open a popover showing its type, status, query, result counts, and a link to the full report; from the popover, activate or deactivate the originating routine without leaving the page.

Inputs and Outputs

- **Inputs:** `GET /api/calendar/events` (executions and scheduled fires) and `GET /api/routines` (routine list used to build the visibility dropdown and to support the popover's activate/deactivate action).
- **Outputs:** no new server-side writes are produced by the page itself, except when the user toggles a routine from the popover, which issues `POST /api/routines/{id}/activate` or `POST /api/routines/{id}/deactivate`.

Known Constraints or Permissions

- Scheduled fires are expanded from each active routine's cron expression via APScheduler's `CronTrigger`. Inactive routines and routines with blank or invalid cron expressions contribute no scheduled events.
- The backend caps scheduled expansion at `MAX_PER_ROUTINE = 200` fires per routine per request and clamps the window to `[now, now + 90 days]` unless FullCalendar supplies an explicit `start` / `end` query string.
- Historical executions are pulled from the local SQLite store via `get_executions(conn, limit=500)`; the 500-row cap means very old executions may not appear once that ceiling is exceeded.
- Past fires are never re-expanded from cron — `window_start` is forced to `now` — so history comes exclusively from the `executions` table.
- Cloud-only executions are not merged in by the current handler; the payload reflects locally recorded executions plus future scheduled fires.

Frontend

Route and Main Component

- Route: `/calendar` (registered in `App.tsx`).
- Main component: `CalendarPage` in `resmon_scripts/frontend/src/pages/CalendarPage.tsx`.

Child Components and Hooks

- `FullCalendar` from `@fullcalendar/react`, configured with `dayGridPlugin`, `timeGridPlugin`, and `interactionPlugin`. A `useRef<FullCalendar>` named `calendarRef` is held for imperative access.
- `PageHelp` renders the in-page help panel (`storageKey="calendar"`).
- Shared `apiClient` wrapper from `../api/client` for all HTTP calls.

UI State Model

Local `useState` slots:

- `events: CalendarEvent[]` — payload returned by `/api/calendar/events`.
- `routines: Routine[]` — payload returned by `/api/routines`, used to render the multi-select dropdown and to support the popover's activate/deactivate action.

- `visibleRoutines: Set<number>` — ids of routines whose events are currently shown. Initialized to the full routine id set after each fetch.
- `popover: PopoverData | null` — the currently open event popover (event snapshot plus screen coordinates) or `null` when closed.
- `typeFilter: string` — `''`, `'deep_dive'`, `'deep_sweep'`, or `'routine'`.
- `statusFilter: string` — raw event `status` string (e.g. `completed`, `running`, `failed`, `cancelled`, `scheduled`).
- `routineDropdownOpen: boolean` — open/closed state of the Routines multi-select.

Key Interactions and Events

- **Initial load and re-fetch on dispatch/completion.** On mount, `fetchData()` issues `/api/calendar/events` and `/api/routines` in parallel via `Promise.all`. A second `useEffect` attaches `window` listeners for `resmon:execution-started` and `resmon:execution-completed` (dispatched by the shared `ExecutionContext`) and re-runs `fetchData()` on either event, so a newly dispatched run appears as a `running` marker and later flips to `completed` / `failed` / `cancelled` in place without a manual refresh.
- **Three-bucket event classification.** `bucketOf(e)` maps each event to one of `'deep_dive' | 'deep_sweep' | 'routine' | 'other'`. Any event carrying a `routine_id` is classified as `routine` regardless of its raw `type` (routine runs reuse the `deep_dive` / `deep_sweep` labels from the sweep engine). A raw `type` of `deep_dive` / `dive` maps to `deep_dive`; `deep_sweep` / `sweep` maps to `deep_sweep`.
- **Filtering.** `filteredEvents` hides events whose `routine_id` is not in `visibleRoutines`, then applies the `Type` filter (compared against `bucketOf(e)`) and the `Status` filter (compared against the raw `status`).
- **Routines multi-select.** `toggleRoutineVisibility(id)` flips a single routine's visibility; `selectAllRoutines()` and `selectNoRoutines()` provide bulk toggles. The dropdown stops click propagation so opening it does not close the popover.
- **Event styling.** Each event is passed to `FullCalendar` with a per-bucket class (`calendar-type-dive`, `calendar-type-sweep`, `calendar-type-routine`) and a `color` string supplied by the backend's status-to-colour map. `extendedProps` carry `routine_id`, `execution_id`, `type`, `status`, `query`, `total_results`, and `new_results` for use by the popover.
- **View and navigation mechanics.** `initialView="dayGridMonth"`; the header toolbar exposes `prev,next today` on the left, the title in the centre, and `dayGridMonth,timeGridWeek,timeGridDay` on the right. `editable={false}` and `selectable={false}` disable drag-to-edit and drag-to-select so the calendar is read-only.

- **Popover open/close.** `handleEventClick(info)` stops propagation on `info.jsEvent`, reads `info.el.getBoundingClientRect()` to position the popover at `{ x: rect.left + rect.width / 2, y: rect.bottom + 4 }`, and toggles the popover closed if the same event id is clicked again. The page's outer `<div className="page-content" onClick={closePopover}>` closes the popover on any outside click; `closePopover()` also collapses the Routines dropdown.
- **Popover routine toggle.** `handleToggleActive()` looks up the routine by `popover.event.routine_id`, picks `activate` or `deactivate` based on `r.is_active`, POSTs to `/api/routines/{id}/{action}`, re-fetches, and closes the popover.

Error and Empty States

- `fetchData()` wraps its network calls in a `try` / `catch` that swallows errors silently; the calendar then renders with whatever `events` / `routines` state it already has (empty on first load).
- The filter bar (`.calendar-filters`) is only rendered when `routines.length > 0`, so a fresh install with no routines shows the calendar grid without filter controls.
- When no executions and no active routines exist, `filteredEvents` is empty and FullCalendar renders an empty grid with no error banner.

Backend

API Endpoints

- `GET /api/calendar/events` — returns the combined list of historical executions and expanded upcoming routine fires. Defined in `resmon_scripts/resmon.py`.
- `GET /api/routines` — list of local routines used to populate the Routines visibility dropdown and the popover's activate/deactivate control.
- `POST /api/routines/{routine_id}/activate` and `POST /api/routines/{routine_id}/deactivate` — invoked from the popover to toggle a routine.

Request and Response Patterns

- `GET /api/calendar/events` accepts two optional query parameters, `start` and `end`, both ISO-8601 strings. FullCalendar emits them on view changes; the handler parses them with `datetime.fromisoformat(...)` (after replacing a trailing `Z` with `+00:00`). If either is missing or

unparseable, the window defaults to `now` through `now + 90 days`. `window_start` is clamped to `now` so past fires are never synthesised.

- The response is a plain JSON array of event objects with the following fields: `id`, `title`, `start`, `end`, `color`, `execution_id`, `routine_id`, `type`, `status`, `query`, `total_results`, `new_results`.
- Historical executions come from `get_executions(conn, limit=500)`; each row is passed through `_enrich_execution_row(ex)` before an event is built. The `title` is `f"{type_label} #{ex['id']}{title_suffix}"` where `title_suffix` is `f": {query}"` when a query exists. `end` falls back to `start_time` when `end_time` is null.
- The status colour palette is:
  - `completed` → `#22c55e` (green)
  - `running` → `#3b82f6` (blue)
  - `failed` → `#ef4444` (red)
  - `cancelled` → `#ef4444` (red, matching the `badge-cancelled` palette used elsewhere)
  - `scheduled` → `#f59e0b` (orange, reserved for upcoming routine fires)
  - any other status → `#6b7280` (grey).
- Scheduled fires are generated per active routine: the handler imports `apscheduler.triggers.cron.CronTrigger`, calls `CronTrigger.from_crontab(cron_expr, timezone=timezone.utc)` on each active routine's `schedule_cron`, and iterates `trigger.get_next_fire_time(prev, cursor)` up to `MAX_PER_ROUTINE = 200` times per routine, stopping when `nxt` is `None` or exceeds `window_end`. Each scheduled event has `id = f"routine-{r['id']}-{nxt.isoformat()}"`, `type = "routine"`, `status = "scheduled"`, `execution_id = None`, and a `query` hint built by joining the routine's `parameters["keywords"]` list with `", "` when present.
- Invalid cron expressions (`ValueError`, `TypeError` from `CronTrigger.from_crontab`) and non-JSON `parameters` fields are caught and skipped rather than aborting the response.
- `ImportError` on `apscheduler` is caught so the endpoint still returns historical executions even when the scheduler dependency is unavailable.

Persistence Touchpoints

- Reads: `get_executions(conn, limit=500)` and `get_routines(conn)` over the local SQLite connection returned by `_get_db()`.
- Writes: none from the calendar endpoint itself. Writes only occur when the popover's activate/deactivate action flips `is_active` via the `/api/routines/{id}/activate` and `/api/routines/{id}/deactivate` handlers.

Execution Side Effects

- The endpoint does not dispatch executions, write logs, send email, or touch cloud storage. It is a read-only aggregator of existing execution rows and of computed upcoming fire times from APScheduler's `CronTrigger`.
- Activate/deactivate calls issued from the popover run through the standard routines CRUD path, which under IMPL-R5 mirrors the change into the APScheduler jobstore so the next `GET /api/calendar/events` reflects the updated active set.